

CSCE 606 SOFTWARE ENGINEERING · MODULE 01 · LECTURE NOTES

Why Software Engineering Exists

Programming integrated over time.

MODULE OUTCOMES

- Distinguish programming from software engineering.
- Distinguish essential from accidental complexity.
- Explain why most software cost occurs after first release.
- Explain why adding people to a late project makes it later.
- Judge a practice by whether it reduces the cost of the next change.

1 Programming and engineering

THE DISTINGUISHING VARIABLE IS TIME

Programming produces code that works now, for the author, on the inputs at hand.

Software engineering produces code that still works in three years — after the author has forgotten it, after several other people have modified it, after the requirements changed twice, and after a dependency released a breaking version.

The difference is not scale. A two-hundred-line script that a team depends on for a decade is an engineering artefact. A hundred-thousand-line simulation run once for a paper is not.

Every practice in this course should be judged against one question: **does it reduce the cost of the next change?** Practices that do not are ceremony, however widely they are recommended.

1.1 Where the cost is

Phase	Approximate share	Comment
Initial development	20–40%	The part that gets estimated, scheduled, and budgeted.
Maintenance and evolution	60–80%	Bug fixes, new requirements, dependency updates, platform changes, performance work.
Within maintenance: understanding existing code	The majority of it	Before any change can be made safely, someone must work out what the current code does and why.

Specific percentages vary between studies and between domains. The ordering does not: post-release cost dominates, and comprehension dominates post-release cost.

CODE IS READ FAR MORE OFTEN THAN WRITTEN

A given line is typed once. It is then read during review, during debugging, during every subsequent change to the surrounding code, and by people who have no context on why it exists.

Optimising for writing speed therefore optimises the wrong variable. A dense one-liner that saves two minutes of typing and costs five minutes of comprehension on each of twenty subsequent readings is a substantial net loss, even though it felt efficient at the time.

This single observation underwrites most of the practices in this course: descriptive names, explicit over implicit, boring over clever, comments that explain *why*, and tests that document intent.

Write for whoever reads it next. Statistically that is you, in six months, with no memory of this afternoon.

2 Essential and accidental complexity

Fred Brooks drew the distinction in *No Silver Bullet* (1986), and it remains the most useful single analytical tool in the subject.

	Essential	Accidental
Source	The problem domain itself.	The tools, languages, and processes used to build the solution.
Examples	Tax rules are genuinely complicated; a payroll system cannot be simpler than the law it implements. Distributed consensus is hard because of physics, not because of our code.	Build system configuration, boilerplate, framework ceremony, duplicated logic, awkward dependency management, four ways to do the same thing.
Can it be removed?	No. It can be relocated, encapsulated, or made explicit — not eliminated.	Yes , and this is where engineering effort compounds.
Trend over time	Changes only if the requirements change.	Grows silently unless someone actively attacks it.

DIAGNOSE BEFORE SIMPLIFYING

Effort spent trying to remove essential complexity is wasted, and often harmful: the complexity does not disappear, it relocates somewhere less visible. A system that hides genuine domain complexity behind a deceptively simple interface has made it *surprising* rather than absent, and surprise is expensive.

Effort spent removing accidental complexity compounds, because it reduces the cost of every future change, not just the current one.

So the first question about any difficult system is which kind of complexity you are looking at. 'This is complicated' is a complaint; 'this is accidentally complicated because the build system requires three files to add one module' is a diagnosis with a fix.

Brooks also argued that accidental complexity had already been largely conquered by high-level languages and that no future development would deliver an order-of-magnitude improvement. The first claim proved

wrong in an interesting way — we invented entirely new categories of accidental complexity, in distributed systems, build tooling, and dependency management. The second claim has held up rather well.

3 Brooks's law

$$\text{communication paths} = n(n - 1) / 2$$

Team size	Pairwise channels
3	3
5	10
10	45
20	190
50	1,225

Output grows at best linearly with team size, and only when the work partitions cleanly. Communication overhead grows quadratically. There is therefore a point beyond which adding people reduces total output, and the point arrives sooner than intuition suggests.

WHAT THE LAW SAYS, AND WHAT IT DOES NOT

It says: adding people to a *late* project makes it later. New arrivals must be brought up to speed by exactly the people who are already behind, so throughput drops before it rises — and on a late project there is no time for it to rise.

It does not say that teams cannot grow, that large projects are impossible, or that hiring is futile. Growing a team early, on work that partitions, is ordinary and effective.

The useful content is the diagnosis, not the prohibition. Communication cost is quadratic, so scaling requires *reducing the need to communicate* — which is precisely what modular design, stable interfaces, and good documentation accomplish. Brooks's law is an argument for Module 03, not against hiring.

CONWAY'S LAW

Organisations design systems that mirror their own communication structure. (Conway, 1967.)

Three teams will produce three components. The interfaces that are awkward will be the ones that fall on team boundaries, because those are the ones that required negotiation rather than a conversation.

This is an empirical observation rather than advice, and it is remarkably robust. It can be exploited: decide the architecture you want, then structure teams to match it — the *inverse Conway manoeuvre*.

It also predicts failure. If the team structure does not match the intended architecture, the architecture will drift to match the teams, not the other way round. Microservice migrations that keep a monolithic team structure reliably produce a distributed monolith.

4 On evidence

Software engineering has far more confident advice than it has evidence. Much of what is asserted is one practitioner's experience generalised well past its warrant, and some widely-repeated claims are simply untested.

Claim	Evidence	Source
Small, frequent deployments outperform large, infrequent ones on both speed and stability.	Strong	DORA / <i>Accelerate</i> , multi-year survey research across thousands of organisations.
Version control, continuous integration, and automated deployment improve delivery outcomes.	Strong	Same.
Code review finds defects and spreads knowledge.	Good	Multiple industrial studies; effect size depends heavily on how review is conducted (Module 09).
Choice of programming language or framework is the dominant factor in project outcome.	Weak	Repeatedly studied; effects are small relative to team and process factors.
Test-driven development specifically improves quality or productivity.	Mixed and genuinely contested	Studies disagree. Writing tests clearly helps; that the tests must come <i>first</i> is not well supported (Module 05).
Pair programming justifies its cost.	Mixed	Benefits for knowledge transfer and for difficult problems; costs are real. Context-dependent.

HOW TO READ THE REST OF THIS COURSE

Where evidence is strong, it is cited. Where a practice is convention rather than demonstrated, that is stated. Where competent practitioners disagree, both positions are given rather than one being presented as settled. Be suspicious of confident advice in this field, including the advice here. The appropriate test is always the same: *does this reduce the cost of the next change, in my situation?*

A practice that works for a team of two hundred on a ten-year product may be pure overhead for one person on a six-month project, and the reverse is also true. Context is not an excuse for sloppiness; it is part of the engineering judgement.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Software Engineering at Google — Chapters 1–3	'Programming over time', the distinction from programming, and the Beyoncé rule. This course's framing comes from here.
Fred Brooks — No Silver Bullet (1986, widely available free)	Essential and accidental complexity. Forty years old and still the sharpest piece of analysis in the field.
Conway — How Do Committees Invent? (1968, free)	The original paper. Short, and the argument is better than the slogan.
DORA — State of DevOps research	The most substantial empirical evidence base in software delivery. Read the summaries before accepting any claim about process.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Take a piece of code you wrote more than six months ago and did not document. Time how long it takes to understand it well enough to make a small change. Write down what would have made it faster.
- 2 Choose the open-source project you will contribute to in Project 2. Criteria: active, has a contribution guide, has tests, and you can build it. Build it today and record how long that took and what went wrong.
- 3 For that project, categorise five sources of difficulty you encountered as essential or accidental complexity. Be specific.
- 4 Find a system you use whose architecture visibly reflects its organisation — inconsistent APIs across subsystems is the usual tell. Write a paragraph on what the organisation chart probably looks like.
- 5 Pick one piece of software engineering advice you believe. Search for empirical evidence for it. Write down what you find, including if the answer is nothing.
- 6 Estimate the lifetime cost split for a project you have worked on. How much went into initial development against subsequent change?

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What distinguishes software engineering from programming, and why is it not size?
- 2 Where does most of a software system's cost occur, and what consumes most of that?
- 3 Define essential and accidental complexity, and say why the distinction changes what you do.
- 4 Why is adding people to a late project counterproductive, and what is the useful conclusion to draw from it?

- 5 State Conway's law and give one way to use it deliberately.
- 6 Name one software engineering claim with strong evidence and one that is genuinely contested.